

Curs 3

Ce puteți face cu procesarea limbajului natural?

Procesarea limbajului natural (NLP) cuprinde un set de tehnici pentru a lucra cu documente scrise într-un limbaj natural pentru a atinge multe obiective diferite. Acestea variază de la cele simple, pe care orice dezvoltator le poate implementa, până la cele extrem de complexe, care necesită multă expertiză.

Table 1. Ce tehnica se aplica unui anumit tip de problema.

What You Need	Where To Look
Grouping similar words	Stemming , Splitting Words , Parsing Documents
Finding words with the same meaning	Latent Semantic Analysis
Generating realistic names	Splitting Words
Understanding how much time it takes to read a text	Reading Time
Understanding how difficult to read is a text	Readability of a Text
Identifying the language of a text	Identifying a Language
Generating a summary of a text	SumBasic (word-based), Graph-based Methods: TextRank (relationship-based), Latent Semantic Analysis (semantic-based)
Finding similar documents	Latent Semantic Analysis
Identifying entities (e.g., cities, people) in a text	Parsing Documents
Understanding the attitude expressed in a text	Parsing Documents
Translating a text	Parsing Documents

Procesarea textului

operație	Metodă	Librării Python recomandate
Gruparea cuvintelor similare	Rădăcina cuvintelor (Stemming), Separarea cuvintelor (Splitting Words), Analiza documentelor (Parsing Documents)	NLTK, spaCy, fuzzywuzzy, rapidfuzz, scikit-learn
Găsirea cuvintelor cu același sens	Analiza semantică latentă (Latent Semantic Analysis)	gensim, scikit-learn, spaCy

Generarea de nume realiste	Separarea cuvintelor (Splitting Words)	markovify, faker
Estimarea timpului necesar pentru a citi un text	Timpul de citire (Reading Time)	readtime, textstat
Înțelegerea nivelului de dificultate al unui text	Lizibilitatea textului (Readability of a Text)	textstat, readability
Identificarea limbii unui text	Identificarea limbii (Identifying a Language)	langdetect, langid, fastText
Generarea unui rezumat al unui text	Metoda SumBasic (bazată pe cuvinte), Metode bazate pe grafuri: TextRank (bazată pe relații), Analiza semantică latentă (bazată pe semnificații)	sumy, gensim, spaCy, transformers
Găsirea documentelor similare	Analiza semantică latentă (Latent Semantic Analysis)	gensim, scikit-learn, sentence-transformers
Identificarea entităților (ex. orașe, persoane) într-un text	Analiza documentelor (Parsing Documents)	spaCy, NLTK, StanfordNLP
Înțelegerea atitudinii exprimate într-un text	Analiza documentelor (Parsing Documents)	TextBlob, VADER, transformers
Traducerea unui text	Analiza documentelor (Parsing Documents)	googletrans, transformers (MarianMT, M2M100)

Pentru gruparea **cuvintelor similare** în Python, există mai multe pachete și biblioteci utile.

scikit-learn: include metode pentru gruparea, precum K-Means, cu funcționalități de preprocesare a datelor și implementări eficiente ale algoritmilor de grupare; scikit-learn poate fi folosit pentru gruparea cuvintelor.

Gruparea cuvintelor similare în Python se referă la tehnici de **identificare și clasificare a cuvintelor care sunt apropiate ca formă sau sens.**

Ce ai nevoie să faci	Unde să cauți / Ce metodă să folosești
Gruparea cuvintelor similare	Rădăcina cuvintelor (Stemming), Separarea cuvintelor (Splitting Words), Analiza documentelor (Parsing Documents)

Pentru identificarea cuvintelor similare putem utiliza bibliotecile:

- **NLTK** sau **spaCy** → pentru *stemming* și *lemmatizare* (reducerea cuvintelor la forma de bază);
- **fuzzywuzzy** sau **rapidfuzz** → pentru potrivire aproximativă (*string similarity*);
- **scikit-learn** → pentru vectorizare și grupare (cum ar fi *KMeans clustering* pe reprezentări TF-IDF);
- **gensim** → pentru *Latent Semantic Analysis* (analiza semantică latentă) sau *Word2Vec* (asemănare pe bază de sens).

Python

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
```

Se importa clasele din biblioteca Scikit-learn (sklearn) pentru a realiza un proces de grupare a datelor text.

from sklearn.feature_extraction.text import TfidfVectorizer - importă clasa TfidfVectorizer din modulul feature_extraction.text din biblioteca Scikit-learn.

TfidfVectorizer este folosit pentru a converti o colecție de documente text în matrice TF-IDF (Term Frequency-Inverse Document Frequency). TF-IDF este o măsură a importanței relative a unui cuvânt într-un document în raport cu întreaga colecție de documente.

Această clasă este folosită în preprocesarea datelor text pentru a le pregăti pentru NLP, algoritmi de machine learning, etc..

from sklearn.cluster import KMeans - importă clasa KMeans din modulul cluster din biblioteca **Scikit-learn**.

KMeans reprezintă o implementare a algoritmului K-Means, un algoritm de grupare (clustering) care împarte datele în clustere bazate pe similaritatea între exemplele de date.

Este frecvent utilizat în analiza datelor pentru grupa conversațiile din setul de date în clustere distincte.

Aplicatie: sa se realizeze un program care combină preprocesarea textului cu tehnica de grupare K-Means pentru a identifica teme sau modele în conversațiile dintre utilizatori și chatbot.

TF-IDF se calculează prin înmulțirea a două valori:

- TF (Term Frequency) este frecvența unui termen într-un document.

$TF(t, d) = \text{numarul de aparitii al termenului } t \text{ în documentul } d / \text{numărul total de termeni în documentul } d$

- IDF (Inverse Document Frequency) - importanța generală a unui termen în întregul corpus de documente.

$IDF(t, D) = \log(\text{numarul total de documente } D / \text{numarul de documente care contin termenul } t)$

Aplicatia 1

`pip install numpy`

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
```

```
# conversatii
```

```
conversations = [
```

```
    "Utilizator: Salut! Care sunt orele de funcționare ale restaurantului vostru?",
```

```
    "Chatbot: Buna! Restaurantul nostru este deschis de la 12:00 la 22:00, de luni până sâmbătă.",
```

```
    "Utilizator: Vreau să rezerv o masă pentru două persoane la ora 19:30.",
```

```
    "Chatbot: Sigur! Rezervarea pentru două persoane la ora 19:30 este confirmată. Vă așteptăm cu plăcere.",
```

```
    "Utilizator: Aveți meniul vegetarian disponibil?",
```

```
    "Chatbot: Da, avem opțiuni vegetariene în meniul nostru. Puteți consulta meniul nostru online sau pot să vă ofer câteva recomandări.",
```

```
    "Utilizator: Cum pot să returnez un produs comandat online?",
```

```
    "Chatbot: Pentru returnarea unui produs, vă rugăm să accesați secțiunea 'Returnări' de pe site-ul nostru sau să contactați departamentul nostru de relații cu clienții la numărul de telefon afișat.",
```

"Utilizator: Care sunt știrile importante de azi?",

"Chatbot: Îmi pare rău, nu pot furniza știri în timp real. Cu toate acestea, vă pot redirecționa către site-uri de știri populare sau vă pot oferi o sursă specifică de știri dacă doriți."

"Utilizator: Care sunt metodele de plată acceptate pentru comenzi online?",

"Chatbot: Acceptăm carduri de credit/debit, plată cu PayPal și, în unele cazuri, plata la livrare. Puteți verifica opțiunile exacte la finalizarea comenzii."

"Utilizator: Puteți să-mi oferi informații despre promoțiile actuale?",

"Chatbot: Desigur! În prezent, avem o promoție specială de 50% reducere la toate comenzile online."

]

Lista personalizată de cuvinte stopwords

custom_stopwords = ["de", "salut", "buna", "da", "pentru", "cum", "care"]

Vectorizăm setul de date text folosind TF-IDF și excluzăm cuvintele stopwords

vectorizer = TfidfVectorizer(stop_words=custom_stopwords)

X = vectorizer.fit_transform(conversations)

Aplicăm algoritmul K-Means pentru gruparea conversațiilor

num_clusters = 2

kmeans = KMeans(n_clusters=num_clusters, random_state=42)

kmeans.fit(X)

afișare

print("Conversațiile grupate:")

order_centroids = kmeans.cluster_centers_.argsort()[:, :-1]

terms = vectorizer.get_feature_names_out()

for i in range(num_clusters):

print(f"Cluster {i + 1}:")

for ind in order_centroids[i, :10]: # Afișăm primele 10 cuvinte cheie din fiecare cluster

print(f" {terms[ind]}")

Discutii:

Gruparea cuvintelor este realizată folosind algoritmul K-Means pe baza vectorilor TF-IDF obținuți din setul de date de conversații.

TF-IDF Vectorization: Conversațiile sunt transformate în vectori TF-IDF cu ajutorul clasei `TfidfVectorizer`. Acest proces cuantifică importanța relativă a cuvintelor în cadrul fiecărei conversații și în întregul set de date.

K-Means Clustering: Algoritmul K-Means este aplicat vectorilor TF-IDF pentru a grupa conversațiile în clustere. K-Means este un algoritm de grupare care împarte datele în clustere în funcție de similaritatea între exemplele de date. În cazul nostru, conversațiile sunt grupate pe baza similarității vectorilor TF-IDF.

Modelul este antrenat pe o gamă largă de texte, inclusiv texte în limba română, și încearcă să găsească pattern-uri și relații între cuvinte în baza acestui antrenament.

`kmeans.cluster_centers_.argsort()[:, ::-1]` este utilizată pentru a ordona coordonatele centrelor de clustere obținute de algoritmul K-means, astfel încât să putem identifica termenii cei mai relevanți (cu cele mai mari valori de TF-IDF) pentru fiecare cluster.

`kmeans.cluster_centers_`: este o matrice în care fiecare rând reprezintă un cluster, iar fiecare coloană reprezintă un termen; valorile din această matrice sunt ponderile TF-IDF asociate fiecărui termen pentru centrul fiecărui cluster.

`.argsort()`: este o funcție care returnează indicii care ar ordona valorile din `cluster_centers_` pentru fiecare rând (cluster), în ordine crescătoare. Pentru fiecare cluster, termenii sunt ordonați în funcție de valoarea lor de TF-IDF.

`[:, ::-1]`: - inversează ordinea indicilor obținuți, astfel încât termenii cu cele mai mari valori de TF-IDF să apară primii. `[:, ::-1]` înseamnă „inversează ordinea”, iar `:` se aplică pe fiecare rând (cluster).

→ `kmeans.cluster_centers_.argsort()[:, ::-1]` creează o listă de indici care ordonează termenii de la cei cu cea mai mare valoare TF-IDF la cea mai mică pentru fiecare cluster. Acest lucru permite extragerea termenilor relevanți din fiecare cluster în ordine descrescătoare a importanței.

Dacă folosim KMeans pe **reprezentările TF-IDF ale cuvintelor sau frazelor**, algoritmul caută **asemănări semantice** între ele și le grupează.

Aplicatie → modificăm aplicația precedentă astfel încât matricea TF-IDF și rezultatele clusteringului să fie scrise în fișiere txt.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
import numpy as np
```

```
# datele de intrare
```

```
conversations = [
```

```
    "Utilizator: Salut! Care sunt orele de functionare ale restaurantului vostru?",
```

```
    "Chatbot: Buna! Restaurantul nostru este deschis de la 8 la 22, de luni pana sambata.",
```

```
    "Utilizator: Vreau sa rezerv o masa pentru doua persoane la ora 11.",
```

```
    "Chatbot: Sigur! Rezervarea pentru doua persoane la ora 11 nu este confirmata. Ne pare rau.",
```

```
    "Utilizator: Aveti meniul vegetarian disponibil?",
```

```
    "Chatbot: Da, avem optiuni vegetariene In meniul nostru. Puteti consulta meniul nostru online sau pot sa va ofer cateva recomandari.",
```

```
    "Utilizator: Cum pot sa returnez un produs comandat online?",
```

```
    "Chatbot: Pentru returnarea unui produs, va rugam sa accesati sectiunea 'Returnari' de pe site-ul nostru sau sa contactati departamentul nostru de relatii cu clientii la numarul de telefon afisat.",
```

"Utilizator: Care sunt stirile importante de azi?",

"Chatbot: Imi pare rau, nu pot furniza stiri In timp real. Cu toate acestea, va pot redirectiona catre site-uri de stiri populare sau va pot oferi o sursa specifica de stiri daca doriti.",

"Utilizator: Puteti sa-mi oferi informatii despre promotiile actuale?",

"Chatbot: Desigur! In prezent, avem o promotie speciala de 50% reducere la toate comenzile online. Utilizati codul 'SPECIAL20' la finalizarea comenzii."

]

Lista personalizată de cuvinte stopwords

custom_stopwords = ["de", "salut", "buna", "da", "pentru", "cum", "care"]

Vectorizăm setul de date text folosind TF-IDF și excludem cuvintele stopwords

vectorizer = TfidfVectorizer(stop_words=custom_stopwords)

X = vectorizer.fit_transform(conversations)

stocăm matricea X într-un fișier text

with open('c://2024/mat_X.txt', 'w') as f:

np.savetxt(f, X.toarray(), fmt="%0.5f")

print("Matricea TF-IDF a fost scrisa in 'mat_X.txt'.")

aplicăm algoritmul K-Means pentru gruparea conversațiilor

num_clusters = 2 # Numărul de clustere dorit

kmeans = KMeans(n_clusters=num_clusters, random_state=42)

kmeans.fit(X)

afișăm rezultatele grupării și salvăm într-un fișier

with open('c://2024/cluster_rezultate.txt', 'w') as f:

f.write("Conversatiile grupate:\n")

order_centroids = kmeans.cluster_centers_.argsort()[:, :-1]

terms = vectorizer.get_feature_names_out()

for i in range(num_clusters):

f.write(f"\nCluster {i + 1}:\n")

print(f"Cluster {i + 1}:")

for ind in order_centroids[i, :10]:

f.write(f" {terms[ind]}\n")

print(f" {terms[ind]}")

print("Rezultatele clusteringului au fost salvate în 'cluster_rezultate.txt'.")

TF-IDF se bazează doar pe frecvența cuvintelor: Algoritmul K-Means + TF-IDF analizează numere, nu sensuri.

îmbunătățești rezultatele:

- **normalizare text** (lowercase, fără punctuație, fără etichetele „Utilizator”/, „Chatbot”)→ elimină zgomotul inutil.
 - **Ar trebui o listă completă de stopwords românești**→ exemplu „la”, „în”, „cu”, „este”, „pe”, „vă”, „un”, „o”, „că”, „care”,,
 - **curațare și lematizare text**→ transformăm cuvintele la forma lor de bază („rezervarea” → „rezerva”, „produse” → „produs”).
- utilizare embeddings semantice, nu doar TF-IDF**

NLTK (Natural Language Toolkit): bibliotecă folosită la procesarea limbajului natural și oferă diverse instrumente pentru tokenizare, stemmare, lemmatizare și chiar pentru gruparea cuvintelor.

Python → `from nltk.cluster import KMeansClusterer`

KMeansClusterer

KMeansClusterer din NLTK este folosit pentru a grupa elemente pe baza similarității dintre ele. Este similar algoritmului KMeans din **scikit-learn**, dar oferă mai multă flexibilitate pentru lucrul cu text, deoarece: permite utilizarea unor metrici de similaritate personalizate între punctele din setul de date; funcționează bine cu modele vectoriale specializate pentru text, cum ar fi **distanța de cosinus**.

KMeansClusterer poate fi integrat ușor în fluxuri de pre-procesare de text, cum ar fi **segmentarea și analiza lexicală**.

Exemplu de utilizare 1

```
pip install nltk
pip install numpy
```

```
from nltk.cluster import KMeansClusterer
from sklearn.feature_extraction.text import TfidfVectorizer
import numpy as np
import nltk

# Asigurați-vă că aveți resursele necesare pentru nltk
nltk.download('punkt')

# Setul nostru de date text (conversații)
conversations = [
    "Utilizator: Salut! Care sunt orele de functionare ale restaurantului vostru?",
    "Chatbot: Buna! Restaurantul nostru este deschis de la 8 la 22, de luni pana sambata.",

    "Utilizator: Vreau sa rezerv o masa pentru doua persoane la ora 11.",
    "Chatbot: Sigur! Rezervarea pentru doua persoane la ora 11 nu este confirmata. Ne pare rau.",

    "Utilizator: Aveti meniul vegetarian disponibil?",
    "Chatbot: Da, avem optiuni vegetariene In meniul nostru. Puteti consulta meniul nostru online sau pot sa va ofer cateva recomandari.",

    "Utilizator: Cum pot sa returnez un produs comandat online?",
    "Chatbot: Pentru returnarea unui produs, va rugam sa accesati sectiunea 'Returnari' de pe site-ul nostru sau sa contactati departamentul nostru de relatii cu clientii la numarul de telefon afisat.",

    "Utilizator: Care sunt stirile importante de azi?",
    "Chatbot: Imi pare rau, nu pot furniza stiri In timp real. Cu toate acestea, va pot redirectiona catre site-uri de stiri populare sau va pot oferi o sursa specifica de stiri daca doriti.",

    "Utilizator: Puteti sa-mi oferi informatii despre promotiile actuale?",
    "Chatbot: Desigur! In prezent, avem o promotie speciala de 50% reducere la toate comenzile online. Utilizati codul 'SPECIAL20' la finalizarea comenzii."
]

# Lista personalizată de cuvinte stopword
custom_stopwords = ["de", "salut", "buna", "da", "pentru", "cum", "care"]

# Vectorizăm setul de date text folosind TF-IDF și excludem cuvintele stopword
```

```

vectorizer = TfidfVectorizer(stop_words=custom_stopwords)
X = vectorizer.fit_transform(conversations).toarray() # Convertim la array

# Salvăm matricea X într-un fișier text
with open('c://2024/mat_X.txt', 'w') as f:
    np.savetxt(f, X, fmt="%0.5f")
print("Matricea TF-IDF a fost scrisa in 'mat_X.txt'.")

# Aplicăm KMeansClusterer pentru gruparea conversațiilor
num_clusters = 2
kmeans = KMeansClusterer(num_clusters, distance=nltk.cluster.util.cosine_distance, repeats=25)
clusters = kmeans.cluster(X, assign_clusters=True)

# afișăm rezultatele grupării și salvăm într-un fișier
with open('c://2024/cluster_rezultate.txt', 'w') as f:
    f.write("conversatiile grupate:\n")
    for i in range(num_clusters):
        f.write(f"\nCluster {i + 1}:\n")
        print(f"Cluster {i + 1}:")
        for j, cluster in enumerate(clusters):
            if cluster == i:
                f.write(f" Conversatie: {conversations[j]}\n")
                print(f" Conversatie: {conversations[j]}")
print("Rezultatele clusteringului au fost salvate în 'cluster_rezultate.txt'.")

```

Exemplu de utilizare 2

Sa se scrie un program care preprocesează conversațiile prin tokenizare, eliminarea cuvintelor stopwords și eliminarea punctuației, apoi calculează și afișează cele mai frecvente cuvinte din setul de date text dat.

Principalele operatii din punct de vedere NLP sunt:

Tokenizare: folosind `word_tokenize` din NLTK se imparte fiecare conversație în cuvinte individuale; textul este redus la litere mici folosind `.lower()`.

Eliminarea cuvintelor stopwords: se construiește o listă de cuvinte stopwords pentru limba română folosind `stopwords.words('romanian')`; utilizatorul este solicitat să introducă o listă personalizată de cuvinte stopwords; listele de cuvinte stopwords implicite sunt combinate într-o singură listă `custom_stopwords`.

Cuvintele stopwords sunt eliminate din conversații pentru a obține `filtered_conversations`.

Eliminarea punctuației: se folosește `re.sub` pentru a înlocui orice caracter care nu este o literă sau spațiu cu un spațiu gol (" "). In conversatie raman doar literele și spațiile.

Calcularea frecvenței cuvintelor: se construiește obiectul `FreqDist` pentru a calcula frecvența cuvintelor în conversații; toate cuvintele sunt colectate în lista `all_words`, iar apoi se calculează frecvența.

Afișarea celor mai frecvente cuvinte: se selectează cele mai frecvente 10 cuvinte folosind `.most_common(10)` pe obiectul `FreqDist`; afisam apoi lista cu cele mai frecvente cuvinte și frecvența lor asociată.

Important

- **from nltk.tokenize import word_tokenize:** importă funcția `word_tokenize` din modulul `nltk.tokenize`. Funcția `word_tokenize` este utilizată pentru a împărți un text în cuvinte individuale, numite tokenuri.
- **from nltk.corpus import stopwords:** Importă modulul `stopwords` din `nltk.corpus`. Acest modul furnizează o listă predefinită de cuvinte stopword pentru diverse limbi, inclusiv limba română.
- **from nltk.probability import FreqDist:** Importă clasa `FreqDist` din modulul `nltk.probability`. Această clasă este utilizată pentru a calcula frecvența cuvintelor într-un set de date text.
- **from nltk import FreqDist:** Importă din `nltk` clasa `FreqDist`. Aceasta este aceeași clasă importată anterior din modulul `nltk.probability`. Această linie poate fi considerată redundantă, deoarece clasa a fost deja importată anterior.
- **import nltk:** Această linie importă întregul modul `nltk`. De obicei, se folosește pentru a aduce întregul set de funcționalități oferite de NLTK într-un program.

Rezolvare

```
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.probability import FreqDist
from nltk import FreqDist
import nltk
#nltk.download('punkt')
#nltk.download('stopwords')
```

import re

Setul nostru de date text (conversații)

conversations = [

"Utilizator: Salut! Care sunt orele de functionare ale restaurantului vostru?",

"Chatbot: Buna! Restaurantul nostru este deschis de la 8 la 22, de luni pana sambata.",

"Utilizator: Vreau sa rezerv o masa pentru doua persoane la ora 11.",

"Chatbot: Sigur! Rezervarea pentru doua persoane la ora 11 nu este confirmata. Ne pare rau.",

"Utilizator: Aveti meniul vegetarian disponibil?",

"Chatbot: Da, avem optiuni vegetariene In meniul nostru. Puteti consulta meniul nostru online sau pot sa va ofer cateva recomandari.",

"Utilizator: Cum pot sa returnez un produs comandat online?",

"Chatbot: Pentru returnarea unui produs, va rugam sa accesati sectiunea 'Returnari' de pe site-ul nostru sau sa contactati departamentul nostru de relatii cu clientii la numarul de telefon afisat.",

"Utilizator: Care sunt stirile importante de azi?",

"Chatbot: Imi pare rau, nu pot furniza stiri In timp real. Cu toate acestea, va pot redirectiona catre site-uri de stiri populare sau va pot oferi o sursa specifica de stiri daca doriti."

"Utilizator: Puteti sa-mi oferi informatii despre promotiile actuale?"

"Chatbot: Desigur! In prezent, avem o promotie speciala de 50% reducere la toate comenzile online. Utilizati codul 'SPECIAL20' la finalizarea comenzii."

]

Lista de cuvinte stopword în limba română

default_stopwords = set(stopwords.words('romanian'))

Solicităm utilizatorului să introducă o listă specifică de cuvinte stopword

user_stopwords = input("Introduceți lista de cuvinte stopword separate prin spații: ")

user_stopwords = set(user_stopwords.split())

combinam lista implicită de cuvinte stopword cu cea introdusă de utilizator

custom_stopwords = default_stopwords.union(user_stopwords)

înlăturăm cuvintele stopword și tokenizăm conversațiile

tokenized_conversations = [word_tokenize(conversation.lower()) for conversation in conversations]

filtered_conversations = [

[word for word in tokens if word not in custom_stopwords] for tokens in tokenized_conversations

]

Eliminăm cuvintele stopword și tokenizăm conversațiile

tokenized_conversations = [word_tokenize(re.sub(r'^a-zA-Z\s', '', conversation.lower())) for conversation in conversations]

filtered_conversations = [

[word for word in tokens if word not in custom_stopwords] for tokens in tokenized_conversations

]

Calculăm frecvența cuvintelor

all_words = [word for tokens in filtered_conversations for word in tokens]

fdist = FreqDist(all_words)

Selectăm cele mai frecvente cuvinte

most_common_words = fdist.most_common(10)

Afișăm cele mai frecvente cuvinte

print("Cele mai frecvente cuvinte:")

for word, frequency in most_common_words:

print(f"{word}: {frequency}")

Gensim: bibliotecă specializată pentru procesarea limbajului natural, inclusiv pentru modelarea tematică și gruparea cuvintelor. Gensim este folosit și pentru prelucrarea documentelor mari.

```

from gensim import corpora, models

# List of documents
documents = [
    "This is the first text",
    "This text is the second text",
    "And this is the third one",
    "Is this the first text",
]

# Tokenize the documents
tokens = [doc.lower().split() for doc in documents]

# Create a dictionary and a corpus
dictionary = corpora.Dictionary(tokens)
corpus = [dictionary.doc2bow(text) for text in tokens]

# Train the LDA model
lda_model = models.LdaModel(corpus, num_topics=2, id2word=dictionary)

# Print the topics
for topic in lda_model.print_topics():
    print(topic)

```

Python `from gensim.models import Word2Vec`

spaCy: bibliotecă pentru procesarea limbajului natural care include funcționalități pentru gruparea cuvintelor și oferă metode pentru obținerea de vectori de cuvinte semantice.

Python `import spacy`

fastText: librărie dezvoltată de Facebook pentru a efectua încorporări de cuvinte (word embeddings) și poate fi folosită pentru gruparea cuvintelor în spațiul vectorial.

Python `import fasttext`

Affinity Propagation: este disponibil în pachetul scikit-learn și poate fi folosit pentru gruparea automată a datelor, inclusiv a cuvintelor.

Python `from sklearn.cluster import AffinityPropagation`

K-Means cu spații vectoriale de cuvinte (Word Embeddings): se folosește K-Means în combinație cu încorporări de cuvinte (word embeddings) pentru gruparea cuvintelor pe baza semnificației lor semantice.

python

Exemplu: utilizarea spațiilor vectoriale de cuvinte cu spaCy și K-Means

import spacy

from sklearn.cluster import KMeans

```
nlp = spacy.load('en_core_web_md') # model lingvistic spaCy
words = ['word1', 'word2', 'word3', ...]
word_vectors = [nlp(word).vector for word in words]
```

```
kmeans = KMeans(n_clusters=3)
kmeans.fit(word_vectors)
labels = kmeans.labels_
```

Modelele lingvistice sunt reprezentări matematice sau statistice ale limbajului natural, concepute pentru a ajuta calculatoarele să înțeleagă, să genereze sau să proceseze limbajul uman. Aceste modele sunt esențiale în domeniul Procesării Limbajului Natural (NLP) și sunt utilizate pentru o varietate de sarcini, cum ar fi **recunoașterea vocală, traducerea automată, analiza de sentiment, generarea de limbaj natural etc.**

Tipuri comune de modele lingvistice:

1. **Modelul de N-gram:** este o secvență de N cuvinte consecutive dintr-un text. Modelele de N-gramuri utilizează frecvențele de apariție ale acestor secvențe pentru a calcula probabilități și pentru a anticipa cuvintele următoare într-un text. Sunt simple și eficiente, dar pot avea dificultăți în a capta dependențe mai lungi între cuvinte.
2. **Modelul de Markov Ascuns (HMM - Hidden Markov Model):** HMM este o tehnică statistică ce presupune că există o "stare ascunsă" (hidden state) care evoluează în timp și care generează observații (cuvinte). Acest model este folosit în recunoașterea vocală și în analiza de limbaj.
3. **Modele de Încorporări de Cuvinte (Word Embeddings):** reprezintă cuvintele sub formă de vectori densi într-un spațiu multidimensional, astfel încât cuvinte similare să fie reprezentate printr-un spațiu asemănător. Modele precum Word2Vec, GloVe și FastText sunt exemple de modele de încorporări de cuvinte.
4. **Rețele Neuronale Recurente (RNN - Recurrent Neural Networks):** sunt rețele neuronale speciale pentru lucrul cu date secvențiale. Ele pot să țină cont de contextul anterior în procesarea limbajului, păstrând o "memorie" a informațiilor anterioare. RNN-urile pot avea dificultăți în a gestiona dependențele pe termen lung.
5. **Rețele Neuronale cu Atenție (Attention Mechanism):** permit modelului să se concentreze asupra părților relevante ale unei intrări într-un anumit moment. Aceasta îmbunătățește capacitatea modelelor de a gestiona dependențele pe termen lung.
6. **Transformatori (Transformer Models):** este o arhitectură de rețea neurală care a devenit extrem de populară pentru sarcini NLP. Modelul BERT (Bidirectional Encoder Representations from Transformers) este un exemplu notabil de transformator folosit pentru sarcini precum învățarea reprezentărilor de cuvinte contextuale.

Relația dintre NLP și gruparea cuvintelor similare, Stemming, Splitting Words, Parsing Documents se referă la diferite aspecte ale prelucrării limbajului natural.

În Procesarea Limbajului Natural (NLP), parsing-ul se referă în principal la analiza și înțelegerea structurilor gramaticale ale limbajului uman.

În NLP, parsarea poate avea mai multe niveluri:

parsarea sintactică (syntactic parsing): analiza structurilor sintactice ale propozițiilor sau frazelor, identificând părțile de vorbire, relațiile sintactice și structura gramaticală generală. Rezultatul poate fi un arbore sintactic care reprezintă relațiile gramaticale dintre cuvinte.

parsarea semantică (semantic parsing): înțelegerea semnificației propozițiilor, a relațiilor semantice dintre cuvinte și a intențiilor vorbitorului. Se încearcă traducerea structurilor sintactice în reprezentări semantice.

parsarea de documente (document parsing): consta în analiza și structurarea informațiilor în documente, identificând entități cheie, relații și contextul general al documentului.

Parsarea se bazează adesea pe gramatici formale, care sunt seturi de reguli care specifică cum pot fi combinate simbolurile pentru a forma propoziții sau structuri mai complexe. Această analiză gramaticală poate fi efectuată manual, dar, în general, este realizată în mod automat folosind algoritmi și tehnici de învățare automată în cadrul NLP.

Un exemplu de parsare sintactică este reprezentat de transformarea unei propoziții într-un arbore sintactic care indică relațiile gramaticale dintre cuvinte, cum ar fi subiectul, predicatul, obiectul etc.

Scopul NLP este să permită calculatoarelor să înțeleagă, să interpreteze și să genereze limbaj uman. NLP utilizează diverse sarcini, inclusiv **gruparea cuvintelor similare, analiza sentimentului, traducerea automată și altele.**

Gruparea Cuvintelor Similare: Gruparea cuvintelor similare este o sarcină specifică din domeniul NLP care implică clasificarea cuvintelor în grupuri sau categorii bazate pe similarități semantice sau contextuale. Aceasta poate fi realizată utilizând modele de învățare automată sau tehnici de încorporare a cuvintelor.

Stemming: este un proces utilizat pentru redarea unui cuvânt la rădăcina sa sau la forma sa de bază. Scopul este de a reduce cuvintele la forma lor principală pentru a realiza o formă comună a aceluiași cuvinte. De exemplu, "bunătațe" și "bun" ar avea aceeași rădăcină, "bun". Stemming-ul ajută la reducerea variabilității cuvintelor și poate spori eficacitatea anumitor sarcini NLP.

Împărțirea Cuvintelor (Tokenization): este procesul de divizare a unui text în unități mai mici, numite token-uri, care pot fi cuvinte sau chiar părți ale cuvintelor. Tokenizarea este un pas esențial în prelucrarea limbajului natural, deoarece transformă textul într-o formă mai ușor de manipulat pentru analiza ulterioară.

Analiza Documentelor (Document Parsing): implică descompunerea și înțelegerea conținutului unui document. Acest lucru poate include identificarea și extragerea informațiilor cheie, detectarea structurilor gramaticale și a relațiilor dintre diferitele elemente ale documentului.

Tehnici de procesare a limbajului natural

1. Sentiment Analysis
2. Named Entity Recognition
3. Summarization

4. Topic Modeling
5. Text Classification
6. Keyword Extraction
7. **Lemmatization** and stemming

1. Analiza sentimentelor

Aceasta este disecția datelor (text, voce, imagini) pentru a determina dacă sunt pozitive, neutre sau negative.



Analiza sentimentelor poate transforma arhive mari de feedback ale clienților, recenzii sau reacții social media în rezultate acționabile și cuantificate. Aceste rezultate pot fi apoi analizate pentru o perspectivă asupra clienților și rezultate strategice suplimentare. <https://monkeylearn.com/sentiment-analysis-online/>

2. Recunoașterea entității numite

Named Entity Recognition, sau NER este o tehnică de procesare a limbajului natural care etichetează "identitățile numite" în text și le extrage pentru o analiză ulterioară.

După cum puteți vedea în exemplul de mai jos, NER este similar cu analiza sentimentelor. Cu toate acestea, NER etichetează pur și simplu identitățile, indiferent dacă sunt nume de organizații, persoane, substantive proprii, locații etc. și ține evidența de câte ori apar într-un set de date.

Ousted **WeWork** founder **Adam Neumann** lists his **Manhattan** penthouse for **\$37.5 million**

[organization] [person] [location] [monetary value]

De câte ori apare o identitate (adică un anumit lucru) în feedback-ul clienților poate indica necesitatea de a **remedia o anumita problema**. În cadrul recenziilor și căutărilor, acesta poate indica o preferință pentru anumite tipuri de produse, permițându-vă să personalizați fiecare călătorie a clientului pentru a se potrivi utilizatorului individual, îmbunătățind astfel experiența clienților.

3. Rezumatul textului

Rezumarea textului este defalcarea jargonului, fie științific, medical, tehnic sau de altă natură, în termeni săi cei mai de bază folosind procesarea limbajului natural pentru a-l face mai ușor de înțeles.

Acest lucru ar putea părea descurajant – limbile noastre sunt complicate. Dar, prin aplicarea algoritmilor de bază de legare **substantiv-verb**, software-ul de rezumat al textului poate sintetiza rapid un limbaj complicat pentru a genera o ieșire concisă. Pentru a crea o aplicație simplă în Python care să facă rezumatul unui text, putem utiliza biblioteca `gensim`. Această bibliotecă oferă funcționalități pentru extragerea de cuvinte cheie și rezumarea textelor folosind algoritmul `TextRank`. Iată un exemplu simplu de aplicație: Pentru a crea o aplicație simplă în Python care să facă rezumatul unui text, putem utiliza biblioteca `gensim`. Această bibliotecă oferă funcționalități pentru extragerea de cuvinte cheie și rezumarea textelor folosind algoritmul `TextRank`.

```
pip install gensim
```

```
from gensim.summarization import summarize
```

```
def main():
```

```
    # Text lung pentru a demonstra rezumarea
```

```
    text = """ Clonarea digitală umană se referă la crearea unei replici virtuale a unei persoane, intr-o entitate care imită aspectul fizic al individului real , personalitatea, comportamentele si chiar vocea. În cazul clonării digitale umane, scopul principal este a se reusi realizarea realizarea unei clone virtuale a unei persoane care, prin interacțiuni sociale, educative sau de divertisment sa evolueze în timp pe baza datelor colectate despre persoana reală. Un exemplu este aplicația Replika, unde utilizatorii pot crea o clonă digitală cu care să interacționeze. Un alt exemplu este Soul Machines care este o companie care creează „ființe digitale” interactive ce pot învăța și interacționa cu oamenii într-un mod natural, simulând emoții și răspunsuri umane.
```

```
    """
```

```
    # Rezumatul textului
```

```
    try:
```

```
        summary = summarize(text, ratio=0.3) # 30% din textul original va fi inclus în rezumat
```

```
        if summary: # Verificăm dacă rezumatul nu este gol
```

```
            print("Rezumatul textului:")
```

```
            print(summary)
```

```
        else:
```

```
            print("Rezumatul nu a putut fi generat.")
```

```
    except ValueError as e:
```

```
        print(f"Error: {e}")
```

```
if __name__ == "__main__":  
    main()
```

Biblioteca sumy este o bibliotecă Python folosită pentru a efectua rezumate de texte. Contine instrumente pentru a sintetiza informațiile cheie dintr-un text și a crea un rezumat coerent.

Caracteristici:

Sumy include diferiți algoritmi de sumarizare, cum ar fi LSA (Latent Semantic Analysis), LexRank, Luhn, etc.

- oferă funcționalități pentru tokenizarea textului și parsarea documentelor în diverse formate.
- permite configurarea parametrilor algoritmilor pentru a obține rezultate personalizate.
- suport pentru diverse limbi și instrumente pentru gestionarea specificităților lingvistice.

Instrucțiunea **from sumy.parsers.plaintext import PlaintextParser** permite pregătirea textului pentru procesul de sumarizare și importă PlaintextParser din modulul plaintext al pachetului sumy.parsers.

PlaintextParser este utilizat pentru a analiza și pregăti un document text pentru sumarizare astfel încât primește textul și un obiect Tokenizer pentru a segmenta textul în unități mai mici, ca de exemplu în propoziții.

Instrucțiunea **from sumy.summarizers.lsa import LsaSummarizer** importă clasa LsaSummarizer din modulul lsa. Iată o detaliere a acestei instrucțiuni:

Principalele Caracteristici ale LsaSummarizer:

Analiza Semantică Latentă (LSA): este o tehnică matematică care explorează relațiile semantice latente dintre termeni într-un set de documente.

Matricea Termeni-Document: algoritmul LSA construiește o matrice termeni-document și aplică metode de reducere a dimensionalității pentru a extrage informații semantice esențiale.

Eliminarea Informațiilor Redundante: LSA este conceput pentru a reduce redundanța în informații, păstrând totuși semnificația esențială.

Instrucțiunea **from sumy.nlp.tokenizers import Tokenizer** importă clasa Tokenizer din modulul tokenizers și permite accesul la clasa Tokenizer pentru a crea un obiect Tokenizer care poate fi utilizat pentru a segmenta textul.

Clasa Tokenizer în sumy: este responsabilă cu împărțirea unui text în unități semantice mai mici, numite "token-uri".

Aplicatia 4

```
from sumy.parsers.plaintext import PlaintextParser  
from sumy.summarizers.lsa import LsaSummarizer
```

```
from sumy.nlp.tokenizers import Tokenizer
```

```
# Textul pe care vrei să-l rezumi
```

```
text = """
```

```
User: Hello! What are your restaurant's opening hours?
```

```
Chatbot: Hello! Our restaurant is open from 12:00 to 22:00, Monday to Saturday.
```

```
User: I want to book a table for two people at 19:30.
```

```
Chatbot: Sure! The reservation for two people at 19:30 is confirmed. We look forward to welcoming you.
```

```
User: Do you have vegetarian menu available?
```

```
Chatbot: Yes, we have vegetarian options on our menu. check out our online menu or I can give you some recommendations.
```

```
User: How can I return a product ordered online?
```

```
Chatbot: To return a product, please go to the 'Returns' section on our website or contact our customer relations department at the phone number shown.
```

```
User: What's the big news today?
```

```
Chatbot: Sorry, I can't provide real-time news. However, they may redirect you to popular news sites or provide you with a specific news source if they receive it.
```

```
User: What payment methods are accepted for online orders?
```

```
Chatbot: We accept credit/debit cards, PayPal payment and in some cases cash on delivery. to verify exact options at checkout.
```

```
User: Give me information about current promotions?
```

```
Chatbot: Of course! We currently have a special promotion of 20% off all online orders. Use code 'SPECIAL20' at checkout.
```

```
"""
```

```
# se creeaza un parser pentru text
```

```
parser = PlaintextParser.from_string(text, Tokenizer("english"))
```

```
# Initializam un sumarizator LSA (Latent Semantic Analysis)
```

```
summarizer = LsaSummarizer()
```

```
# Obținem rezumatul
```

```
summary = summarizer(parser.document, 3) # Numărul de propoziții în rezumat
```

```
# afisare
```

```
for sentence in summary:
```

```
    print(sentence)
```

4. Modelarea subiectelor este o tehnică nesupravegheată de procesare a limbajului natural care utilizează programe de inteligență artificială pentru a eticheta și grupa grupuri de text care împărtășesc subiecte comune.

5. Clasificarea textului este organizarea unor cantități mari de text nestructurat (adică datele text brute pe care le primiți de la clienții dvs.). Modelarea subiectelor, analiza sentimentelor și extragerea cuvintelor cheie (pe care le vom parcurge în continuare) sunt subseturi ale clasificării textului.

Clasificarea textului preia setul de date text, apoi îl structurează pentru o analiză ulterioară. Este adesea folosit pentru a extrage date utile din recenziile clienților, precum și din jurnalele de servicii pentru clienți.

6. Extragerea cuvintelor cheie - extragerea cuvintelor cheie este procesul automat de extragere a celor mai relevante informații din text folosind AI și algoritmi de învățare automată.

Exemplu: <https://monkeylearn.com/keyword-extractor-online/>

7. Lemmatizarea și stemarea - sunt două tehnici de preprocesare în Procesarea Limbajului Natural (NLP) care au ca scop simplificarea cuvintelor la forma lor de bază (cunoscută și sub denumirea de formă canonizată sau lemat). Cu toate că ambele tehnici au un obiectiv similar, există diferențe între ele în modul în care efectuează această simplificare.

Lemmatizarea este o tehnică în Procesarea Limbajului Natural (NLP) care constă în aducerea cuvintelor la forma lor de bază sau la lema lor, respectând regulile limbii. Această tehnică poate fi preferată în diverse scenarii din NLP, în funcție de nevoile și obiectivele specifice ale aplicației.

Se poate spune ca, lemmatizarea este procesul de reducere a unui cuvânt la forma sa de bază sau la forma sa de dicționar, numita "lemă".

Exemplu: Pentru cuvântul "better", lema este "good". Lemmatizarea va transforma "better" în "good".

Cand folosim lematizarea:

Dacă păstrarea sensului exact al cuvintelor este esențială într-o anumită situație, atunci este de preferat a se utiliza lematizarea față de stemare. **Lematizarea furnizează cuvinte care sunt leme valide în limbajul natural, ceea ce facilitează înțelegerea semnificației.**

În construirea de modele de limbaj sau modele de clasificare bazate pe cuvinte, lematizarea poate ajuta la reducerea variabilității și la concentrarea pe informațiile semantice de bază.

În procesul de preprocesarea textelor pentru analiza de sentiment unde se explorează semnificația cuvintelor în context, lematizarea poate oferi o reprezentare mai precisă a cuvintelor cheie.

Lematizarea se bazează pe baze de date lexicale, cum ar fi **WordNet**, pentru a identifica lemele.

În dezvoltarea de dicționare sau indecși de cuvinte pentru prelucrarea textelor, lematizarea poate ajuta la organizarea și structurarea datelor într-un mod semnificativ.

În aplicații de căutare de informații, unde precizia este esențială, lematizarea poate contribui la eliminarea redundanței și la îmbunătățirea relevanței rezultatelor.

Pentru operațiile care implică relații gramaticale între cuvinte, cum ar fi analiza sintactică, lematizarea poate ajuta la stabilirea corelațiilor semantice între cuvinte.

Important: Lematizarea poate fi mai costisitoare din punct de vedere computațional decât stemarea și există scenarii în care alegerea între cele două tehnici depinde tipul de problema la care se aplica.

Stemarea (stemming) este o tehnică utilizată în preprocesarea textelor în cadrul Procesării Limbajului Natural (NLP). Este important să înțelegi că stemarea este o metodă mai agresivă de simplificare a cuvintelor la forma lor de bază (sau la un "stem"), fără a lua în considerare sensul lor semantic. Decizia de a utiliza stemarea depinde de obiectivele specifice ale aplicației NLP și de cerințele proiectului.

Exemplu: Pentru cuvântul "running", stemul este "run". Stemarea va transforma "running" în "run".

Lematizarea oferă întotdeauna un cuvânt valid, deoarece se bazează pe lemele din dicționar.

Stemarea poate oferi un "stem" care nu este întotdeauna un cuvânt valid, deoarece este o simplificare mai brută.

Lematizarea ține cont de contextul lingvistic pentru a oferi o formă de bază validă.

Stemarea nu ține cont de context și trunchiază cuvintele pentru a obține o formă de bază.

Lematizarea poate fi mai computațional intensivă decât stemarea, deoarece implică analiza lingvistică a cuvintelor în context.

Stemarea este, în general, mai rapidă, dar poate furniza rezultate mai brute.

Ambele tehnici sunt utile în procesul de preprocesare a textelor în NLP, iar alegerea dintre lematizare și stemare depinde de obiectivele specifice ale proiectului și de contextul în care sunt folosite.

Exemple pentru care operația de stemarea poate fi utilă:

Stemarea poate ajuta la reducerea dimensiunii vocabularului prin eliminarea sufixelor și prefixelor, ceea ce poate facilita procesul de analiză a textelor într-un mod mai eficient.

La căutarea pe internet sau în bazele de date, stemarea poate accelera procesul de căutare, deoarece reduce cuvintele la forma lor de bază și face căutările să fie mai eficiente.

Uneori pentru anumite task-uri precum gruparea sau clasificarea documentelor, este util să ai cuvinte reduse la forma lor de bază pentru a evidenția similaritățile între texte.

Daca se lucrează cu texte mari sau în timp real, stemarea poate fi preferată pentru a reduce complexitatea analizei textuale.

În modelarea de limbaj sau analiza de sentimente, unde sensul exact al cuvintelor poate să nu fie atât de important, stemarea poate oferi un compromis între simplificare și conservarea informației.

Atentie: Din cauza naturii sale agresive, stemarea poate duce la stemuri care nu sunt cuvinte valide sau care au sens redus.

`from nltk.stem import PorterStemmer, WordNetLemmatizer` - face parte din biblioteca nltk (Natural Language Toolkit) și importă două clase esențiale pentru procesarea limbajului natural (NLP): PorterStemmer și WordNetLemmatizer.

PorterStemmer este un algoritm de stemare, adică o metodă de reducere a cuvintelor la forma lor de bază sau la o formă radical simplificată. Este utilizat pentru a elimina sufixele cuvintelor pentru a obține forma lor redusă (stem).

WordNetLemmatizer: este folosit pentru lematizare, adică aducerea unui cuvânt la forma sa de bază sau la lema sa, respectând regulile limbii. Este mai avansat decât stemarea și încearcă să ofere forma corectă și completă a cuvântului.

nltk.download('punkt'): descarcă resursele necesare pentru tokenizare în nltk. Tokenizarea este procesul de împărțire a unui text în unități semantice mai mici, numite "token-uri", cum ar fi cuvintele sau simbolurile de punctuație.

nltk.download('wordnet'): descarcă resursele necesare pentru lematizare în nltk. Lematizarea implică aducerea unui cuvânt la forma sa de bază sau la lema sa într-un limbaj natural.

WordNetLemmatizer din nltk.stem utilizează o bază de date lexicală numită **WordNet** pentru a realiza lematizarea. Descărcarea resurselor prin nltk.download('wordnet') asigură că această bază de date este disponibilă pentru a efectua corect operațiile de lematizare.

Discutii cod:

```
# Stemarea cuvintelor
stemmer = PorterStemmer()
stemmed_words = [stemmer.stem(word) for word in words]
```

stemmer = PorterStemmer() creează o instanță a clasei PorterStemmer din biblioteca nltk.

PorterStemmer este un algoritm de stemare dezvoltat de Martin Porter și este larg utilizat în procesarea limbajului natural.

Aplicatie in care se aplica # Tokenizare cuvintelor, # Stemarea cuvintelor si # Lematizarea cuvintelor

```
import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer
```

```

from nltk.tokenize import word_tokenize
nltk.download('punkt')
nltk.download('wordnet')

# Text de exemplu
text = """SpaceX CEO Elon Musk has shared another picture of his
company's new space suits, this time showing off a full-body shot of what the outfit will look
like.
Shared on his Instagram, the image shows someone wearing the suit while standing next to a
Crew Dragon capsule,
the vehicle that SpaceX is building to take NASA astronauts to and from the International
Space Station
for the agency's Commercial Crew Program"""

# Tokenizare cuvintelor
words = word_tokenize(text)

# Stemarea cuvintelor
stemmer = PorterStemmer()
stemmed_words = [stemmer.stem(word) for word in words]

# Lematizarea cuvintelor
lemmatizer = WordNetLemmatizer()
lemmatized_words = [lemmatizer.lemmatize(word) for word in words]

# Afişare rezultate
#print("Text original:", words)
print("Cuvinte stemate:", stemmed_words)
#print("Cuvinte lematizate:", lemmatized_words)

```

rezultatele lematizării sunt cuvinte de bază valide, în timp ce rezultatele stemării sunt cuvinte trunchiate, care pot să nu fie întotdeauna cuvinte valide.

Tema 1

```
import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
import re

# Descărcăm resursele necesare (documentați dacă nu le aveți deja)
# nltk.download('punkt')
# nltk.download('wordnet')
# nltk.download('stopwords')

# Text de exemplu
text = """SpaceX CEO Elon Musk has shared another picture of his company's new space suits, this time showing off
a full-body shot of what the outfit will look like.
Shared on his Instagram, the image shows someone wearing the suit
while standing next to a Crew Dragon capsule,
the vehicle that SpaceX is building to take NASA astronauts
to and from the International Space Station for the agency's Commercial Crew Program"""

# Curățarea textului (eliminarea caracterelor nealfanumerice și transformarea în litere mici)
text_nou = re.sub(r'[^\w\s]', '', text.lower())

# Tokenizare cuvintelor
words = word_tokenize(text_nou)

# Afișăm cuvintele tokenizate
print("Cuvinte tokenizate:", words)

# Crearea setului de stopwords
default_stopwords = set(stopwords.words('english'))
print("Default stopwords:", default_stopwords)

# Filtrarea cuvintelor pentru a elimina stopwords
filtered_words = [word for word in words if word not in default_stopwords]
print("Cuvinte filtrate:", filtered_words)

# Stemarea cuvintelor
stemmer = PorterStemmer()
stemmed_words = [stemmer.stem(word) for word in filtered_words]

# Lematizarea cuvintelor
lemmatizer = WordNetLemmatizer()
lemmatized_words = [lemmatizer.lemmatize(word) for word in filtered_words]

# Afișare rezultate
print("Text original:", words)
print("Cuvinte stemate:", stemmed_words)
print("Cuvinte lematizate:", lemmatized_words)
```

tema 2

```
import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
import re

#nltk.download('punkt')
#nltk.download('wordnet')
#nltk.download('stopwords')
# Text de exemplu
def curata_tweeturi(tweeturi):
    tweeturi_curate = []

    stop_words = set(stopwords.words('english'))

    for tweet in tweeturi:
```

```

# Tokenizare cuvinte
cuvinte = word_tokenize(tweet)

# Eliminare hashtag-uri, @tweets, link-uri/ caractere speciale
cuvinte_curate = [cuvant for cuvant in cuvinte if cuvant not in stop_words and
cuvant.isalnum()]

# Se reconstruieste textul din cuvintele curatate
tweet_curat = ' '.join(cuvinte_curate)

# tweet-ul curat la lista
tweeturi_curate.append(tweet_curat)

return tweeturi_curate
text_initial = "SpaceX CEO Elon Musk has shared another picture of his company's
new space suits, 1000 #buna this time showing off a full-body shot of what the outfit will
look like. Shared on his Instagram, the image shows someone wearing the suit while
standing next to a Crew Dragon capsule, the vehicle that SpaceX is building to take
NASA astronauts to and from the International Space Station for the agency's
Commercial Crew Program"
# Pastram din text doar caracterele alfanumerice
text = re.sub("[^a-zA-Z\s]", "", text_initial)
# Tokenizare cuvintelor
text = curata_tweeturi(text)
#words = word_tokenize(text)
#print("Cuvinte curatate:", text)
# Stemarea cuvintelor
stemmer = PorterStemmer()
stemmed_words = [stemmer.stem(word) for word in words]

# Lematizarea cuvintelor
lemmatizer = WordNetLemmatizer()
lemmatized_words = [lemmatizer.lemmatize(word) for word in words]

# Afişare rezultate
#print("Text original:", words)
print("Cuvinte stemate:", stemmed_words)
print("Cuvinte lematizate:", lemmatized_words)

```